torch.sparse_csc_tensor#

https://pytorch.org/docs/stable/generated/torch.sparse_csc_tensor.html#torch

Description:

Constructs a sparse tensor in CSC (Compressed Sparse Column) with specified values at the given ccol_indices and row_indices. Sparse matrix multiplication operations in CSC format are typically faster than that for sparse tensors in COO format. Make you have a look at the note on the data type of the indices. If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor. ccol_indices (array_like) – (B+1)-dimensional array of size (*batchsize, ncols + 1). The last element of each batch is the number of non-zeros. This tensor encodes the index in values and row_indices depending on where the given column starts. Each successive number in the tensor subtracted by the number before it denotes the number of elements in a given column. row_indices (array_like) – Row coordinates of each element in values. (B+1)-dimensional tensor with the same length as values.

Function Signature

```
torch.sparse_csc_tensor(ccol_indices, row_indices, values,
size=None, *, dtype=None, device=None, pin_memory=False,
requires_grad=False, check_invariants=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, infers data type from values. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False. check_invariants (bool, optional) – If sparse tensor invariants are checked. Default: as returned by torch.sparse.check_sparse_tensor_invariants.is_enabled

Code Examples

```
>>> ccol_indices = [0, 2, 4]
>>> row_indices = [0, 1, 0, 1]
>>> values = [1, 2, 3, 4]
>>> torch.sparse_csc_tensor(torch.tensor(ccol_indices,
dtype=torch.int64),
...                         torch.tensor(row_indices,
dtype=torch.int64),
...                         torch.tensor(values),
dtype=torch.double)
tensor(ccol_indices=tensor([0, 2, 4]),
       row_indices=tensor([0, 1, 0, 1]),
       values=tensor([1., 2., 3., 4.]), size=(2, 2), nnz=4,
       dtype=torch.float64, layout=torch.sparse_csc)
```

Notes & Warnings

> **NOTE** Note If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

Detailed Documentation

Documentation

Constructs a sparse tensor in CSC (Compressed Sparse Column) with specified values at the given ccol_indices and row_indices. Sparse matrix multiplication operations in CSC format are typically faster than that for sparse tensors in COO format. Make you have a look at the note on the data type of the indices.

If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

ccol_indices (array_like) – (B+1)-dimensional array of size (*batchsize, ncols + 1). The last element of each batch is the number of non-zeros. This tensor encodes the index in values and row_indices depending on where the given column starts. Each successive number in the tensor subtracted by the number before it denotes the number of elements in a given column.

row_indices (array_like) – Row coordinates of each element in values. (B+1)-dimensional tensor with the same length as values.

values (array_list) – Initial values for the tensor. Can be a list, tuple, NumPy ndarray, scalar, and other types that represents a (1+K)-dimensional tensor where K is the number of dense dimensions.

size (list, tuple, torch.Size, optional) – Size of the sparse tensor: (*batchsize, nrows, ncols, *densesize). If not provided, the size will be inferred as the minimum size big enough to hold all non-zero elements.

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, infers data type from values.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()).

device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

check_invariants (bool, optional) – If sparse tensor invariants are checked. Default: as returned by torch.sparse.check_sparse_tensor_invariants.is_enabled(), initially False.
